

Capítulo 6: Pattern Matching y Tipos de Datos Algebraicos (ADTs) Básicos

Este capítulo aborda las herramientas que definen el estilo de programación moderno y funcional de Scala. El **Pattern Matching** (Coincidencia de Patrones) no es solo una estructura de control avanzada, sino que, cuando se combina con los **Tipos de Datos Algebraicos (ADTs)**, permite modelar el dominio de negocio de manera que se maximiza la seguridad de tipos y se previenen errores lógicos en tiempo de compilación.

1. Explicación Teórica: El Binomio ADT y Pattern Matching

Para el desarrollador que viene de Java o Python, el concepto central es que Scala te obliga a modelar tus datos con una precisión tal que la lógica de negocio se vuelve trivial.

Pattern Matching (Coincidencia de Patrones)

El Pattern Matching, implementado con la palabra clave `match`, es la herramienta más potente de flujo de control en Scala, superando las capacidades del `switch` o de las anidadas condiciones `if-else` en otros lenguajes.

En esencia, el `match` es una **expresión** que evalúa un valor, comparándolo contra una serie de *patrones* definidos secuencialmente. Estos patrones no solo se limitan a valores literales, sino que pueden deconstruir objetos complejos y extraer sus componentes internos.

Tipos de Datos Algebraicos (ADTs)

Un ADT, en Scala, es la forma idiomática de modelar un dominio encapsulando el **estado inmutable** de una entidad. La estructura fundamental de un ADT es la siguiente:

- Tipo de Suma (Sum Type):** Se utiliza una clase padre (anteriormente `sealed trait` en Scala 2) o, preferiblemente, la nueva sintaxis `enum` de Scala 3. Esto define el conjunto **finito** de todos los posibles estados o variantes que puede tomar una entidad (por ejemplo, un `EstadoDePago` puede ser *Pendiente* o *Completado*). El uso de `sealed` o `enum` asegura que el compilador conozca todas las posibles variantes, lo que permite la verificación de exhaustividad.
- Tipo de Producto (Product Type):** Se utiliza la `case class` para modelar los datos inmutables asociados a cada variante.

2. Sintaxis y Ejemplos de Código (ADTs y Match)

Utilizaremos un ejemplo real: modelar los comandos que un sistema de actores (como Pekko) podría recibir.

A. Definición del ADT con `enum` (Tipo de Suma)

Definimos el tipo base `ComandoInventario` con la palabra clave `enum` en Scala 3. Cada caso dentro del `enum` es un tipo de producto que encapsula los datos específicos que necesita.

```
1 // El enum define el tipo base Command, que puede ser cualquiera de sus
  // casos.
2 enum ComandoInventario:
3   // Caso 1 (Producto): Encapsula datos para añadir un artículo
4   case AñadirArticulo(id: String, cantidad: Int)
5   // Caso 2 (Producto): Encapsula datos para retirar stock
6   case RetirarStock(id: String, cantidad: Int)
7   // Caso 3 (Objeto Singleton): No necesita datos, solo la acción
8   case GenerarReporte
9
10 // Uso en instanciación (gracias a case class/enum, no se usa 'new'):
11 val comandoAñadir = ComandoInventario.AñadirArticulo("ABC-123", 100)
12 val comandoReporte = ComandoInventario.GenerarReporte
```

B. Pattern Matching para la Lógica de Negocio

El Pattern Matching se utiliza para analizar la estructura de un valor `ComandoInventario` y ejecutar la lógica apropiada, garantizando que todos los casos definidos en el `enum` sean cubiertos (chequeo de exhaustividad).

```
1 /**
2  * Procesa un comando de inventario y devuelve una descripción de la
3  * acción.
4  * Note el retorno implícito de 'String' (expresión).
5  */
6 def procesar(comando: ComandoInventario): String =
7   // La estructura 'match' se usa en el valor 'comando'
8   comando match
9     // 1. Coincidencia de Patrón por tipo y desestructuración:
10    //   Extrae el 'id' y la 'cantidad' de la case class AñadirArticulo.
11    case ComandoInventario.AñadirArticulo(idArticulo, num) =>
12      s"Añadiendo $num unidades del artículo ID: $idArticulo."
13
14    // 2. Coincidencia de Patrón con Guardas (Guards):
15    //   Solo coincide si la cantidad a retirar es mayor que cero.
16    case ComandoInventario.RetirarStock(id, cantidad) if cantidad > 0 =>
17      s"Retirando $cantidad unidades del ID: $id. Acción válida."
18
19    // 3. Coincidencia de Patrón por valor (Singleton Object):
20    case ComandoInventario.GenerarReporte =>
```

```

20     "Iniciando generación de reporte de inventario."
21
22     // 4. Caso Wildcard (coincide con todo lo demás, opcional si el enum
    es exhaustivo):
23     case _ =>
24     "Comando o estado de datos no reconocido." // Solo se usa si no se
    confía en la exhaustividad

```

3. Características Avanzadas de Pattern Matching

La potencia del Pattern Matching va más allá de la simple selección de comandos:

A. Desestructuración de Nidos (Deep Matching)

Permite extraer valores de estructuras anidadas de forma concisa.

```

1  case class Usuario(id: Long, info: DatosContacto)
2  case class DatosContacto(email: String, telefono: Option[String])
3
4  val usuarioComplejo = Usuario(201, DatosContacto("test@mail.com",
    Some("555-1234")))
5
6  val resultado = usuarioComplejo match
7    // Coincide si el ID es 201 y si el teléfono (anidado en info) es
    Some(valor)
8    case Usuario(201, DatosContacto(_, Some(numTelefono))) =>
9      s"Usuario 201 tiene teléfono: $numTelefono" // numTelefono es el
    String dentro de Some
10
11    // Coincide con cualquier Usuario que no tenga teléfono o no sea el 201
12    case _ => "Teléfono no disponible o usuario incorrecto"
13
14    // resultado: "Usuario 201 tiene teléfono: 555-1234"

```

B. Coincidencia por Tipo (Type Matching)

Se puede utilizar para comparar el tipo de una expresión y acotar su alcance, similar a lo que haría `instanceof` en Java, pero capturando y transformando el valor inmediatamente.

```

1  def describir(x: Any): String =
2    x match
3      case s: String => s"Cadena de tamaño ${s.length}"
4      case l: List[_] => s"Lista con ${l.size} elementos" // l se trata como
    List[_]
5      case n: Int if n < 0 => "Número entero negativo" // Tipado y guarda a
    la vez

```

4. Comparativa con Java y Python

Característica	Java (Switch Statement/If-Else)	Python (Estructuras Dinámicas)	Scala (Match Expression / ADT)
Modelado de Variantes	Interfaces o Clases abstractas. El uso de <code>instanceof</code> o campos <code>type</code> es manual y propenso a errores.	Clases dinámicas. No hay chequeo en compilación.	<code>enum</code> / <code>sealed trait</code> garantiza la exhaustividad.
Control de Flujo	<code>switch</code> (solo en valores primitivos o enums) o anidación de <code>if-else</code> .	Múltiples <code>if/elif</code> .	<code>match expression</code> , se asigna directamente a un <code>val</code> inmutable.
Seguridad/Exhaustividad	El compilador NO garantiza que se cubran todos los subtipos de una jerarquía de clases.	No aplica (dinámico).	El compilador advierte si falta un <code>case</code> para un subtipo del <code>enum</code>.
Desestructuración	Requiere el uso manual de <i>getters</i> en las clases de datos (salvo Pattern Matching de Java 17+ limitado a <code>instanceof</code> /records).	Extracción manual de campos.	Desestructura automáticamente (<code>case Nombre(campo1, campo2)</code>).

5. Best Practices (*The Scala Way*)

El uso de ADTs y Pattern Matching es la base de la programación funcional robusta en Scala.

- Imponer la Exhaustividad:** Utilizar `enum` (o `sealed trait` en Scala 2) como la raíz del ADT. Esto permite que el compilador garantice el chequeo de **exhaustividad**: si un nuevo caso se añade al `enum` (por ejemplo, `ComandoInventario.Cancelar`), el compilador forzará al desarrollador a actualizar todos los `match` expressions que utilicen ese `enum`.
- Case Classes por Defecto:** Utilizar `case class` para todos los tipos de producto (las variantes del `enum`). Esto proporciona los métodos automáticos de `equals`, `copy` y, fundamentalmente, el método `unapply` que habilita la desestructuración eficiente en el Pattern Matching.

3. **Modelar el Dominio, No la Lógica:** Utilizar Pattern Matching para **modelar la lógica de negocios** (las decisiones), reservando el ADT para modelar **el estado** (las posibles formas del dato).
4. **Uso Idiomático:** El `match` debe ser usado como una **expresión**, asignando siempre su resultado a un valor inmutable (`val`), en lugar de utilizarlo para ejecutar efectos secundarios dispersos (instrucciones).

El Pattern Matching y los ADTs funcionan como un seguro de vida para el código: al obligar al desarrollador a declarar explícitamente todos los estados posibles de un dato (`enum`), el compilador puede actuar como un inspector de aduanas que verifica que el código maneje cada posible "forma" del dato, eliminando los errores de tiempo de ejecución causados por estados no previstos.